编程实验 3

题目：

采用栈的方式求一个表达式的值, 例:2+4*(7-5)= ?

参考：https://blog.csdn.net/weixin_43638873/article/details/115675639

提示：以上链接中实现的表达式比较复杂，我们这里只考虑较简单的情况，即 a+b*(c-d)，其中 a,b,c,d 均为整数。

要求：采用顺序栈和链栈分别实现。

源程序：

顺序栈实现：



```c
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

#define MAX 100

// 定义顺序栈
typedef struct {
    int data[MAX];
    int top;
} Stack;

// 初始化栈
void initStack(Stack* stack) {
    stack->top = -1;
}

// 判断栈是否为空
int isEmpty(Stack* stack) {
    return stack->top == -1;
}

// 入栈
void push(Stack* stack, int value) {
    if (stack->top < MAX - 1) {
        stack->data[++stack->top] = value;
    }
    else {
        printf("栈上溢\n");
    }
}

// 出栈
int pop(Stack* stack) {
    if (!isEmpty(stack)) {
        return stack->data[stack->top--];
    }
    else {
        printf("栈下溢\n");
        return -1;
    }
}

// 获取栈顶元素
int peek(Stack* stack) {
    if (!isEmpty(stack)) {
        return stack->data[stack->top];
    }
    return -1;
}

// 判断操作符优先级
int precedence(char op) {
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    return 0;
}

// 执行运算
int applyOperation(int a, int b, char op) {
    switch (op) {
    case '+': return a + b;
    case '-': return a - b;
    case '*': return a * b;
    case '/': return a / b;
    }
    return 0;
}

// 计算表达式的值
int evaluateExpression(const char* expression) {
    Stack values, ops;
    initStack(&values);
    initStack(&ops);

    for (int i = 0; expression[i] != '\0'; i++) {
        // 忽略空格
        if (expression[i] == ' ') continue;

        // 如果是数字，推入数值栈
        if (isdigit(expression[i])) {
            int val = 0;
            while (i < strlen(expression) && isdigit(expression[i])) {
                val = (val * 10) + (expression[i] - '0');
                i++;
            }
            push(&values, val);
            i--;  // 退回一步，因为外层循环会增加i
        }
        // 左括号直接推入运算符栈
        else if (expression[i] == '(') {
            push(&ops, expression[i]);
        }
        // 右括号则计算直到左括号
        else if (expression[i] == ')') {
            while (!isEmpty(&ops) && peek(&ops) != '(') {
                int val2 = pop(&values);
                int val1 = pop(&values);
                char op = pop(&ops);
                push(&values, applyOperation(val1, val2, op));
            }
            pop(&ops);  // 弹出左括号
        }
        // 遇到操作符
        else {
            while (!isEmpty(&ops) && precedence(peek(&ops)) >= precedence(expression[i])) {
                int val2 = pop(&values);
                int val1 = pop(&values);
                char op = pop(&ops);
                push(&values, applyOperation(val1, val2, op));
            }
            push(&ops, expression[i]);
        }
    }

    // 计算栈中剩余的值
    while (!isEmpty(&ops)) {
        int val2 = pop(&values);
        int val1 = pop(&values);
        char op = pop(&ops);
        push(&values, applyOperation(val1, val2, op));
    }

    return pop(&values);
}

int main() {
    const char expression[] = "2 + 4 * (7 - 5)";
    printf("顺序栈实现: 表达式%s = %d\n", expression, evaluateExpression(expression));
    return 0;
}
```

链栈实现：

```c
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

typedef struct Node {
    int data;
    struct Node* next;
} Node;

// 入栈
void push(Node** top, int value) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = value;
    newNode->next = *top;
    *top = newNode;
}

// 出栈
int pop(Node** top) {
    if (*top == NULL) {
        printf("栈下溢\n");
        return -1;
    }
    int value = (*top)->data;
    Node* temp = *top;
    *top = (*top)->next;
    free(temp);
    return value;
}

// 获取栈顶元素
int peek(Node* top) {
    if (top != NULL) {
        return top->data;
    }
    return -1;
}

// 判定操作符优先级
int precedence(char op) {
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    return 0;
}

// 执行运算
int applyOperation(int a, int b, char op) {
    switch (op) {
    case '+': return a + b;
    case '-': return a - b;
    case '*': return a * b;
    case '/': return a / b;
    }
    return 0;
}

// 计算表达式的值
int evaluateExpression(const char* expression) {
    Node* values = NULL;
    Node* ops = NULL;

    for (int i = 0; expression[i] != '\0'; i++) {
        if (expression[i] == ' ') continue;

        if (isdigit(expression[i])) {
            int val = 0;
            while (i < strlen(expression) && isdigit(expression[i])) {
                val = (val * 10) + (expression[i] - '0');
                i++;
            }
            push(&values, val);
            i--;
        }
        else if (expression[i] == '(') {
            push(&ops, expression[i]);
        }
        else if (expression[i] == ')') {
            while (ops != NULL && peek(ops) != '(') {
                int val2 = pop(&values);
                int val1 = pop(&values);
                char op = pop(&ops);
                push(&values, applyOperation(val1, val2, op));
            }
            pop(&ops);
        }
        else {
            while (ops != NULL && precedence(peek(ops)) >= precedence(expression[i])) {
                int val2 = pop(&values);
                int val1 = pop(&values);
                char op = pop(&ops);
                push(&values, applyOperation(val1, val2, op));
            }
            push(&ops, expression[i]);
        }
    }

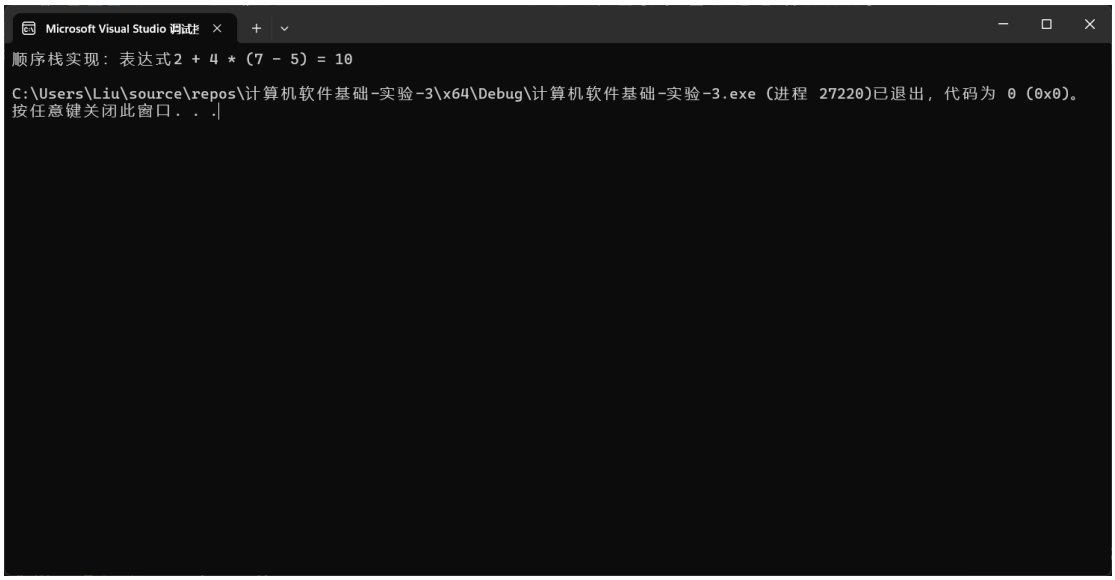
    while (ops != NULL) {
        int val2 = pop(&values);
        int val1 = pop(&values);
        char op = pop(&ops);
        push(&values, applyOperation(val1, val2, op));
    }

    return pop(&values);
}

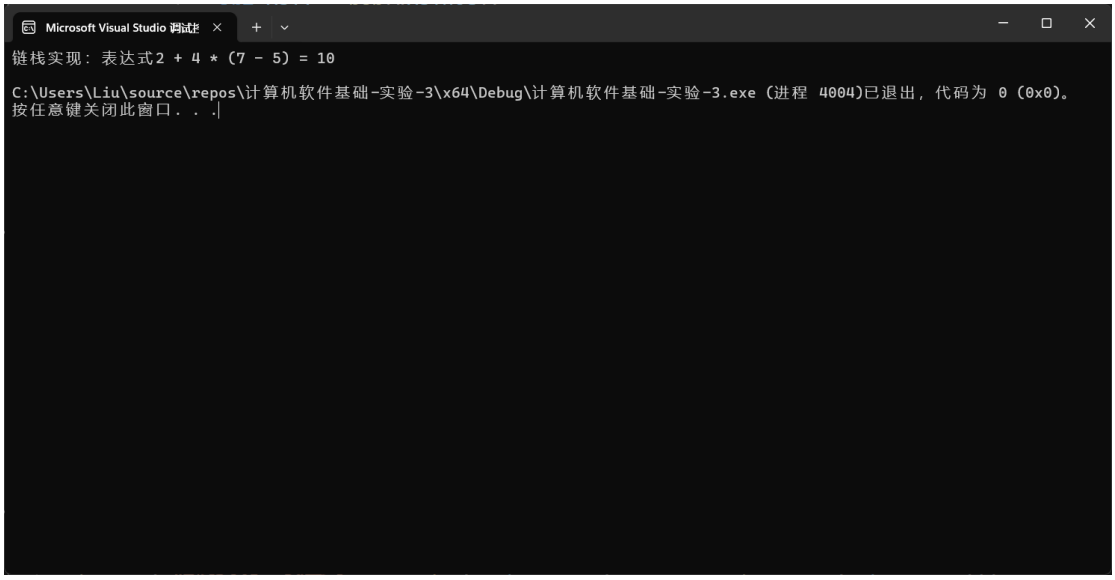
int main() {
    const char expression[] = "2 + 4 * (7 - 5)";
    printf("链栈实现：表达式%s = %d\n", expression, evaluateExpression(expression));
    return 0;
}
```

运行结果：

顺序栈实现：



链栈实现：



心得体会：

通过这次编程实验，我深刻理解了栈在表达式求值中的应用。实现顺序栈和链栈的过程中，我不仅熟悉了栈的基本操作，也体会到不同实现方式的优缺点。同时，学习了将中缀表达式转换为后缀表达式的方法，简化了复杂的优先级和括号处理。实际编码让我意识到细节和调试的重要性，遇到的逻辑错误让我更加细致地思考程序的严谨性。理论知识在实际编程中的应用带给我很大的成就感，也加深了我对算法和数据结构的理解，这次实验大大帮助我提升了实际编程能力和解决问题的思维。